

Artificial Intelligence in Pla...

Member-only story

5 Claude Code Agents You Can Build in less than 10 Minutes

Are You Aware Of The Subagent Architecture?



Shashwat

Follow

5 min read · May 18, 2026

156

1





Photo by [Sander Sammy](#) on [Unsplash](#)

Before I begin,

| *Writing boilerplate PRs, checking test coverage, hunting dead code, and*

parsing server logs are not high-value tasks.

They are kinda like friction, tbh.

And If you are doing these manually every single day, your money making tendency is broken.

. . .

When I was auditing the backend data pipelines for GritGlean, I realized *my token burn and mental bandwidth were being wasted on repetitive terminal commands.*

Developers recently shared a massive technical teardown exposing how they automated all of this, which I bluntly copied.

These are the best five among those which can be done in less than 10 minutes.

. . .

Most Importantly,

they stopped dumping massive logs into their main Claude Code session and started building isolated, single-file AI agents.

So, how Claude Code agents actually work, the exact YAML configurations for 5 autonomous agents you can deploy in under 10 minutes is below:

. . .

Firstly, building an agent is entirely file-based.

You do not need a complex SaaS platform.

You simply create a markdown file inside your `.claude/agents/` directory with a YAML frontmatter block.

Claude Code automatically reads the description and auto-delegates the task when it matches your prompt, or you can manually invoke it by typing `@agent-name`.

By pushing these tasks to isolated subagents, you keep your main

session context clean and drastically reduce your API costs.

Here are the 5 deployment configurations.

1. The PR Summarizer

Reading a branch diff and manually formatting a pull request is a waste of human compute.

This agent reads your local git history and outputs a clean, structured PR description ready to paste directly into GitHub.

File: `.claude/agents/pr-summarizer.md`

```
---
name: pr-summarizer
description: Generate a PR description from current branch changes.
model: claude-sonnet-4-5
tools:
  - Read
  - Grep
  - Glob
  - Bash
---

1. Run `git log main..HEAD --oneline` to get all commits.
2. Run `git diff main...HEAD --stat` for changed files summary.
3. Read the key changed files to understand full context.

Generate PR description in this format:
## What
[One paragraph: what this PR does]
## Why
[One paragraph: why this change is needed]
```

```
## Changes
[Bullet list of key changes grouped by area]
## Testing
[How this was tested]
Output ready to paste into GitHub. Nothing else.
```

2. The Test Coverage Engine

Instead of blindly writing tests, you instruct an agent to run your test suite, analyze the coverage gaps, and prioritize the exact functions that require attention.

File: `.claude/agents/coverage-checker.md`

```
---
name: coverage-checker
description: Analyze test coverage and find untested code.
model: claude-sonnet-4-5-20250929
tools:
  - Read
  - Bash
  - Grep
  - Glob
---
1. Run the test suite with coverage.
2. Parse the coverage report.
3. Identify files with lowest coverage.
4. For each low-coverage file, read the source and identify untested func

Output:
## Coverage Summary
Total: [X]% | Statements: [X]% | Branches: [X]%
## Lowest Coverage Files
[file]: [X]% missing tests for [specific functions]
```

```
## Recommended Next Tests
Priority 1: [file] [function] (handles [critical path])
Priority 2: [file] [function] (handles [error case])
Keep recommendations specific and actionable.
```

3. The Dead Code Sweeper

Orphaned files and unused exports accumulate rapidly in large codebases. This agent recursively scans your project for unused logic and generates a clean removal hitlist.

File: `.claude/agents/dead-code.md`

```
---
name: dead-code
description: Find unused code, unreachable functions, and orphaned files.
model: claude-sonnet-4-5-20250929
tools:
  - Read
  - Grep
  - Glob
---

1. Find all exported functions and classes across the codebase.
2. For each export, grep for imports and usage in other files.
3. Identify exported functions never imported, files never imported, and c

Output:
## Unused Exports
[file]: [export name] not imported anywhere
## Orphaned Files
[file] never imported by any other file
## Cleanup Candidates
[file]: [description of what can be removed]
Be conservative. If uncertain, mark as "verify before removing."
```

4. The Database Migration Generator

Writing raw SQL or ORM migrations manually introduces critical syntax risks. This agent reads your existing schema conventions and generates the precise UP and DOWN migrations autonomously.

File: `.claude/agents/migration-gen.md`

```
---
name: migration-gen
description: Generate database migrations. Use when adding tables or changing schema.
model: claude-sonnet-4-5-20250929
tools:
  - Read
  - Write
  - Glob
  - Bash
---

1. Read existing migrations to understand the tool, naming convention, and schema.
2. Read the current schema file.
3. Create the migration file following project conventions.

Rules:
* Always include both UP and DOWN migrations.
* Add indexes for foreign keys and frequently queried columns.
* Never add NOT NULL without a DEFAULT value.
After creating, run the migration locally and verify it applies cleanly.
```

5. The Error Log Analyzer

Dumping a 10,000 line server log into your primary Claude window

will obliterate your context limit. This agent runs in a dedicated container, filters the noise, and returns only the root causes.

File: `.claude/agents/error-analyzer.md`

```
---
name: error-analyzer
description: Analyze error logs and identify patterns.
model: claude-sonnet-4-5-20250929
tools:
  - Read
  - Bash
  - Grep
---
1. Read the log file filtering for ERROR and WARN only: `grep -n 'ERROR\|WARN' [log_file]`
2. Group errors by type and pattern.
3. For each group, identify frequency, first/last occurrence, stack trace

Output:
## Error Summary
Total errors: [N] | Unique patterns: [N] | Time span: [range]
## Pattern 1: [Error Type] (occurred [N] times)
First seen: [timestamp]
Last seen: [timestamp]
Stack: [key frames]
Likely cause: [explanation]
Suggested fix: [action]
Prioritize by frequency and severity.
```

• • •

The primary leverage here is token economics.

By default, these agents are routed to `claude-sonnet-4-5`, which is roughly 5x cheaper than Opus.

- Because each subagent executes in total isolation, the raw terminal output from a massive grep search or a 200 line log file stays trapped inside the agent's specific context window.
- Without agents, parsing an error log, generating a PR, and hunting dead code will dump 300,000 tokens into your main session. With agents, the heavy lifting happens in the background, and your main session only receives the finalized 500 word summary.

You execute the exact same work, but you eliminate 90% of your token waste.

Build the files,

Set your constraints,

Let the agents run :)

• • •

In case we are meeting for the first time, come over here, it'll be worth the roller coaster of articles that are gonna come up in the next few weeks.

I swear tracking these updates is a job in itself, lately.

Here's the list which I've built and keep adding on.

And If you need help for analyzing UFC fights, please check out BoutPredict :)

Software Engineering

Claude Code

Artificial Intelligence

AI Agent

Developer Tools



Published in Artificial Intelligence in Plain English

Follow

44K followers · Last published 3 hours ago

New AI, ML and Data Science articles every day. Follow to join our 3.5M+ monthly readers.



Written by Shashwat

Follow



780 followers · 16 following

Sharing everything I learn (mostly tech). Scaling boutpredict.aibucket.org to 150 paid & 10,000+ free monthly users.
Business Inquiry: send2shashwat@gmail.com



Placeholder for bio text

Placeholder for bio text

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

[Redacted]

